What cap-x Lacks: Gaps Found by Dogfooding a Drawer-Opening Skill on LIBERO-PRO

cap-x-se / self-evolve dogfooding

June 25, 2026 (living draft — derived from `capx-se/open_drawer/`)

Abstract

We probe what the cap-x agent framework is missing by *dogfooding*: hand-authoring a LIBERO drawer-opening skill using only cap-x's legitimate perception + planning vocabulary (agentview / wrist RGB-D + proprioception in, joint actions out; *no* privileged simulator state on the solve path), running it live, and recording every place the stack fell short. The exercise both (a) opens the drawer robustly and safely —**a cold-start runner now succeeds 5/5** (`qpos` $0 \to -0.160$, object disturbance 4–25 mm), up from a 3/5 naive baseline that plowed through the scene (bowl $\sim$210 mm)—and (b) surfaces a coherent set of framework gaps. The headline gap: cap-x has **no deterministic short-range "seat" primitive**; its only motion engine is a stochastic trajectory optimizer whose final-grasp landing height scatters by RNG, so precision grasping is non-repeatable. Beyond that we find a self-inconsistent task/eval contract, a motion stack of half-wired pieces (a collision-aware planner that had never run from the reduced API), an execution model that fights the fixed eval horizon, no contact/collision feedback to the skill, and no headless multi-round execution loop. Legend throughout: **[patched]** fixed this session; **[flag]** needs a decision; **[built]** new tooling added.

1 Setup: the dogfooding contract

The task is the LIBERO `open_the_middle_drawer_of_the_cabinet` scenario (`libero_goal` task 0; success = drawer `qpos` < -0.14). The hard rules (from `AGENT.md`) are the point of the exercise:

- **No privileged state on the solve path.** Object/joint ground truth a real robot cannot read is forbidden; everything is estimated from cameras + proprioception. Privileged reads are allowed *only* in the runner, for scoring.
- **Success is necessary but not sufficient — it must be SAFE.** The skill must not collide with or disturb other objects; the runner reports task success *and* object disturbance. Safety dominates success: fail safely rather than succeed unsafely.
- **No overfitting** to a single task/seed; closed-loop on sensing feedback. Multi-step interaction (drive a segment, observe, drive the next) and the wrist eye-in-hand camera are encouraged.

The legitimate vocabulary — SAM3 text segmentation, depth→3D, Contact-GraspNet, pyroki/CuRobo IK and motion, the blocking joint controller, gripper control — is *sufficient in principle*: from perception alone we detect the three handles, pick "middle" from the instruction, estimate the prismatic axis, grasp, and pull the drawer fully open. The primitives are sound; the gaps are in how they are assembled, observed, and made repeatable.

2 The headline gap: no deterministic short-range seat

cap-x's only motion engine is the HORL/pyroki **trajectory optimizer** (trajopt). For long-range *transit* it is fine. For the final ∼5 cm *seat* — dropping the gripper precisely onto a thin handle bar — it is the wrong tool, and this was the single root cause of unreliable success.

- **Non-repeatable landing.** The *same* seat target lands the TCP at z scattering **0.10–0.17 by RNG seed**. The middle handle is a horizontal D-bracket bar at $z \approx 0.110$; only a vertically centered grip ($z \approx 0.10$) drags the drawer. A landing 1–6 cm high grips *above* the bar and pulls nothing. With a warm planner (RNG advanced) we hit good seats → 5/5 in-harness; with a cold planner (one process per seed) the seat RNG gated success.
- **Untrustworthy cost.** The trajopt intermittently returns NaN final cost (diverged) and a *phantom* $\sim 10^{10}$ collision cost for a gripper passing 7 cm *above* a flat object whose real disturbance is ∼3 mm. The cost cannot be a safety gate.

[patched] (worked around). We replaced the trajopt seat with a **deterministic** `solve_ik` seat: solve IK at the bar pose, then linearly interpolate joints over the pre→bar corridor (verified clear by the sensing point cloud). This lands the TCP repeatably deep and centered (measured z = 0.109/0.110/0.110 over three reps vs the trajopt's 0.10–0.17), and on the cold runner it lands on-bar on the *first try* every seed. Caveat: pyroki IK quality depends on a good warm-start — called from a high arm config it returns a $z \approx 0.14$ solution — so the seat re-descends and re-solves if it lands off-bar.

What cap-x needs: a first-class deterministic short-range seat primitive (direct IK at the grasp pose + collision-checked linear interp over a clear corridor). The planner is right for transit and wrong for the precision seat; the framework should ship both.

3 The task / eval contract is not self-consistent

LIBERO-PRO's `_task` variant perturbs the BDDL. For `open_the_middle_drawer_of_the_cabinet` we found three disagreeing sources of truth (Table 1). `load_libero_task` *deliberately* fed the agent the suite-metadata language ("middle"), but `check_success()` scores the BDDL goal ("bottom"). **Following the instruction can never pass**, and the entire `_task`/`_swap` arm is silently mis-scored. Verified: opening `middle_level` → `check_success() = False`; setting `bottom_level` = −0.16 → `True`.

source	value
filename	`open_the_middle_drawer…`
BDDL `(:language …)`	**"open the bottom drawer of the cabinet"**
BDDL `(:goal …)`	`(Open wooden_cabinet_1_bottom_region)`
suite metadata `task.language`	**"open the middle drawer of the cabinet"**

Table 1: Three disagreeing "truths" for one perturbed task.

[patched] `load_libero_task` now reads the BDDL `(:language …)` (the same source as the scored goal), prefers it, and warns when it diverges from suite metadata. Non-perturbed suites are unaffected. [flag] This flips the instruction shown to the agent across every perturbed task — it needs confirmation that BDDL-as-truth is the intended scoring for the PRO suites. **Framework**

invariant: instruction and reward must come from one source, and the loader should *assert* they agree (or log loudly).

4 The motion stack is a collection of half-wired pieces

A capability the agent is *told* it has but that errors on first call is worse than not having it. The only collision-aware planner (CuRobo) had never run from the reduced API:

1. [patched] **`get_ee_pose` never existed.** `update_curobo_world_with_object` and `plan_with_grasped_object` call `self.get_ee_pose()`, but no class defined it (the in-repo example codes call it too — they could never have run). Added `FrankaLiberoApiReduced.get_ee_pose()` reading `robot_cartesian_pos` (real-robot legitimate).
2. [patched] **warp API drift.** Vendored CuRobo `geom/sdf/world_mesh.py` calls `wp.torch.device_from_torch`, removed in warp 1.14. Every collision-world build raised `AttributeError`. Added a compat shim.
3. [patched] **Not exposed.** `plan_grasp_trajectory` / `plan_with_grasped_object` / `execute_joint_trajectory` / `parse_grasp_poses_for_curobo` were commented out of `functions()`. Exposed once the above made them work.

After the fixes the CuRobo IK path works and is **accurate (∼5 mm)**, far better than pyroki for low grasps. Remaining issues:

- [flag] **Collision-aware reach-into-container fails.** `plan_grasp_trajectory(use_world_collision=True)` returns `GRAPH_FAIL` / "Start or End state in collision" for a handle adjacent to the cabinet mesh (home config flagged in-collision; goal grazes the scene mesh). Workable only collision-off. Needs better scene-collision ignoring / sphere-buffer tuning.
- [flag] **`solve_ik` silently substitutes orientation.** It tries the requested quaternion, then silently falls back to top-down / 45° / side and returns whatever solves — the caller cannot tell which it got. For a directional drawer grasp this breaks Cartesian continuity. Want a `strict=True` mode (raise instead of re-orienting) or return the achieved orientation.
- [flag] **pyroki IK is inaccurate at low/awkward poses.** The soft least-squares solver left ∼5 cm error at the low handle ($z \approx 0.04$), grasping above it; CuRobo IK hit 5 mm. Route precision moves through CuRobo IK, or tighten the pyroki cost/iterations.
- [flag] **RRT to a near-structure goal fails silently — and hangs.** `planner.plan(…)` to a pre-grasp near the cabinet intermittently returns not-planned with "RRTConnect: start tree could not be initialized" (start config reads in-collision against the depth cloud). The caller gets a silent no-op and, unless it *verifies* the achieved pose, proceeds from the wrong place (stranded high → a too-high seat). Worse: after the pull, with the arm parked at the open drawer, RRT reads the *start* as in-collision and **hangs the whole run** though the drawer is already open. We worked around both by retrying/verifying and by using trajopt for the retreat. Needs a reliable near-contact RRT or a documented "verify the move landed" contract.
- [flag] The HTTP CuRobo client (`capx/integrations/motion/curobo.py`) is **dead code**; the pyroki `/plan` route silently ignores obstacles. One motion facade with a tested contract should subsume these, and the dead paths should be deleted.

5 Execution model fights the fixed-horizon eval

[flag] `move_to_joints_blocking` spends up to `max_steps` sim-steps *per waypoint*. `execute_joint_trajectory(subsample=1, max_steps=400)` over a 30-waypoint plan blew the default episode horizon → robosuite "executing action in terminated episode". Conversely the default `max_steps=120` under-converged (joints 0.39 rad short, ∼5 cm off) until raised to ∼250–400, and it stops short *silently*. Needs a horizon-aware (time-parameterized, budget-checked) trajectory executor and an explicit convergence-failure signal the skill can branch on.

6 No contact / collision feedback to the skill

[flag] The bottom drawer fails because the hand-collision box hits the table and the gripper clips the stacked upper handles and a plate — but the skill is **blind** to all of it (we saw it only via privileged `sim.data.contact`). With perception only, the skill cannot distinguish "didn't reach" from "reached but blocked". Expose the proprioceptive collision signals a real robot also has — commanded-vs-achieved pose error, joint-effort / contact spikes, gripper-didn't-close-on-anything — as first-class API outputs, and let the planner plan against the *observed* scene mesh so it routes around obstacles instead of `GRAPH_FAIL`-ing because the goal grazes a mesh.

7 Control mode: position-only, no compliance

[flag] The deepest blocker for the (unreachable) bottom drawer is the control mode. cap-x drives a **blocking `JOINT_POSITION` controller** against a **coarse gripper collision box** (half-extent 0.104). Measured floor: the wrist cannot descend below $z \approx 0.09$ near the table, but the bottom handle is at $z = 0.042$ — so the end-effector physically cannot reach it from the front/side, and top-down is blocked by the stacked upper drawers. Every primitive (CuRobo, pyroki IK, hook/drag, graspnet) hits this same wall. Trained LIBERO policies open it because they use **compliant OSC** that pushes through soft table/cabinet contacts a position controller treats as hard stops. Expose an operational-space / impedance mode (or a contact-tolerant guarded-move primitive); this is likely the single most important missing capability for low/cluttered targets.

8 Grasp synthesis is too weak for thin/cluttered handles

[flag] Contact-GraspNet on a thin handle and a top-down heuristic do not cover a low handle wedged above a table, below stacked handles, with a plate in front; graspnet returns only low-confidence / degenerate grasps there. cap-x also has *no* primitive for a front/horizontal grasp or for estimating an articulation axis — PCA plane-fit on the cabinet points gives the wrong axis ($-X$; truth $+Y$) because depth sees front+top faces. We worked around it by estimating the axis from cabinet-centroid→handle and then *probing* (pull a little, re-observe the handle's 3D displacement). Add a non-prehensile "hook and drag" primitive and integrate grasp selection with the collision world so only *reachable* grasps are proposed. (AnyGrasp + a real collision world is the expected fix; until then, some initial scenes where graspnet finds no viable grasp are simply skipped.)

9 Robustness: a "success" that knocks things over is not a solution

[flag] On the *successful* naive middle-drawer run the arm displaced the bowl ∼210–240 mm, cream cheese ∼100 mm, plate ∼40 mm — a real-robot failure. Causes and findings:

- Collision-off motion plows through the scene; the front-grasp corridor passes right over the bowl in front of the cabinet.
- Collision-aware CuRobo works but is finicky: default robot spheres flag the start in-collision (→`GRAPH_FAIL`); shrinking spheres plans but then the under-sized model grazes obstacles for real. Full-size + larger `robot_distance_threshold` both plans and cuts disturbance to ∼29 mm in isolation.
- But it is non-deterministic in the full pipeline: the joint-space plan to the IK solution (`plan_single_js`) isn't world-collision-checked the way the pose plan is, so the executed path still sometimes grazes the bowl (∼120 mm). The collision guarantee is only as strong as the weakest stage.
- A single-view world mesh (marching cubes from one agentview depth) underestimates partially-occluded objects, so even a correct planner routes too close.
- The pull is not collision-aware/constrained and over-pulls past the drawer limit.

Improvements: (a) one collision-aware facade whose *every* stage (IK seed, joint plan, Cartesian pull) is world-checked; (b) a multi-view (agentview+wrist) *fused* collision world; (c) obstacle-aware grasp/approach selection; (d) object disturbance reported as a first-class eval metric alongside `check_success`, so "robust" is measured, not assumed.

10 Tooling we had to build

- **[built] Headless skill runner.** Every skill ran inside the LLM `exec()` loop; there was no clean "inject `functions()`, run code, report `check_success()`" entrypoint. We built one (`harness.py` / `run_robust.py`) — also the shape of the self-evolve Benchmark Evaluator's `eval_fn` and of regression tests.
- **[built] Multi-round sim-driving loop.** cap-x had no way to "run a segment → observe sensing+GT → write/run the next segment" against a live env while keeping the (expensive, ∼2 min compile) planner warm. We built `interactive.py` + `ictl.sh`: load env + planner once, then exec Python snippets from a file inbox in a persistent namespace with `reset_env(s)` for the reset-on-collision rule. Nearly every design discovery came from it.
- **[built] Depth→collision-world HORL planner** (`horl_planner.py`): HORL RRT+trajopt wrapped for cap-x, collision world built from the agentview depth cloud; fixed horizon + fixed sphere pool ⇒ JAX compiles once then ∼0.6 s/solve.
- **[flag]** Ergonomic foot-guns: the control-API ctor eagerly warms up pyroki (needs the server even for skills that never use IK); the pyroki server has no `/health` route (404); raw obs depth is $(H, W, 1)$ while the reduced API squeezes to (H, W); the in-repo `*_CODE` examples call functions not in the current `functions()` (example rot).

11 What we added over stock cap-x to reach 5/5

The win came from one decisive change plus the supports that let it land safely every time (detail in `DESIGN_SUMMARY.md`):

1. **Deterministic IK seat** (§2) — replaced the stochastic trajopt seat. This is what took success from 3/5 to a deterministic 5/5.

2. **Depth-point-cloud collision world** + HORL planner so transit routes *around* the scene objects instead of through them ($\sim$210 mm $\rightarrow$ tens of mm).
3. **Sensing-based safety gate**, not the planner's internal cost: accept a seat by the executed TCP (`on_bar`) and gripper reading, not by trajopt cost (which false-positives).
4. **Pose-verified descend-to-pre** (RRT silent-fail fallback) and **trajopt retreat** (RRT-hang fix).
5. **Geometry-aware grasp**: recognize the D-bracket bar (perpendicular to the $+Y$ pull), center the grip vertically on the bar, descend through the clear gap between the plate edge and the handle, and tune the pull to stop right at the drawer travel.

12 Prioritized recommendations

1. **One consistent task/eval contract** — single source for instruction + reward, asserted at load (§3).
2. **One tested motion facade with a headless runner** — IK, free-space plan, collision-aware plan, *and a deterministic short-range seat*; CI smoke test that plans+executes one grasp; delete the dead paths (§2, §4).
3. **Contact/collision feedback + observed-scene collision planning** (§6), plus an **operational-space / compliant control mode** (§7).
4. **A small articulated-manipulation atomic library** — `estimate_front_normal`, `front_grasp`, `probe_prismatic_axis`, `pull_open` — exactly the atoms this task needed (§8).
5. **Object disturbance as a first-class eval metric** alongside success (§9).

The perception + planning *ingredients* are good; they are not yet assembled into a reliable, observable, eval-consistent control loop. With the above, both of cap-x's stated north-stars — benchmark accuracy and faster human-in-the-loop success — get materially easier.